Divide the data into train and test

```
X.shape
```

```
(22092, 10)
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
X_train.shape
```

```
(17673, 10)
```

```
X_test.shape
```

```
(4419, 10)
```

Select the model for training

```
RF_model = RandomForestRegressor(random_state=42)
```

Step 4: Training

```
RF_model.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

Step 5 Prediction and Evaluation

```
RF_y_pred = RF_model.predict(X_test)
```

```
RF_y_pred[:20]
```

```
array([2.93080000e-01, 1.00000000e-03, 1.21122793e-03, 1.16130018e-03,
       0.00000000e+00, 4.00000000e-03, 1.24555977e-04, 2.20009044e-03,
       2.00000000e-03, 3.94080000e-01, 0.00000000e+00, 1.40000000e-02,
       4.08395607e-03, 7.00000000e-03, 2.15970231e-03, 2.89160331e-04,
       1.02821706e-03, 3.15790000e-01, 9.00000000e-03, 0.00000000e+00])
```

```
RF_mse = mean_squared_error(y_test, RF_y_pred)
RF_rmse = np.sqrt(RF_mse)
RF_r2 = r2_score(y_test, RF_y_pred)
print(f'RMSE: {RF_rmse}')
print(f'R² Score: {RF_r2}')
```

```
RMSE: 0.006143789217304181
R² Score: 0.9993280085696331
```

```
from sklearn.linear_model import LinearRegression
LR_model = LinearRegression()
```

```
LR_model.fit(X_train, y_train)
```

```
LR_y_pred = LR_model.predict(X_test)
```

```
LR_mse = mean_squared_error(y_test, LR_y_pred)
LR_rmse = np.sqrt(LR_mse)
LR_r2 = r2_score(y_test, LR_y_pred)

print(f'RMSE: {LR_rmse}')
print(f'R² Score: {LR_r2}')
```

```
RMSE: 0.00028073792916293835
R² Score: 0.9999985968848819
```

Step 6: Hyperparameter Tuning

```
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(RandomForestRegressor(random_state=42), param_grid, cv=3, n_jobs=-1)

grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
print("Best Parameters:", grid_search.best_params_)
```

```
Best Parameters: {'max_depth': 20, 'min_samples_split': 2, 'n_estimators': 100}
```

Use best parameters for prediction

```
# Use the best model to make predictions on the test set
y_pred_best = best_model.predict(X_test)


HP_mse = mean_squared_error(y_test, y_pred_best)
HP_rmse = np.sqrt(HP_mse)
HP_r2 = r2_score(y_test, y_pred_best)

print(f'RMSE: {HP_rmse}')
print(f'R² Score: {HP_r2}')
```

```
RMSE: 0.005948528382514106
R² Score: 0.9993700440298772
```

Step 7: Comapartive Study and Slecting the Best model

```
results = {
    'Model': ['Random Forest (Default)', 'Linear Regression', 'Random Forest (Tuned)'],
    'MSE': [RF_mse, LR_mse, HP_mse],
    'RMSE': [RF_rmse, LR_rmse, HP_rmse],
    'R2': [RF_r2, LR_r2, HP_r2]
}

comparison_df = pd.DataFrame(results)
print(comparison_df)
```

```
                     Model           MSE      RMSE        R2
0  Random Forest (Default)  3.774615e-05  0.006144  0.999328
1        Linear Regression  7.881378e-08  0.000281  0.999999
2    Random Forest (Tuned)  3.538499e-05  0.005949  0.999370
```

Save model and encoders

```
!mkdir models
```

```
mkdir: cannot create directory ‘models’: File exists
```

```
joblib.dump(best_model, 'models/LR_model.pkl')
joblib.dump(scaler, 'models/scaler.pkl')
```

```
['models/scaler.pkl']
```